

PDF Question-Answering Agent Design and Evaluation Report

SRH University Berlin
Large Language Models, Prompting, and
Agentic AI

1. The task and why it fits agent automation

The business process automated here is document question-answering: a user supplies a PDF and asks natural-language questions about its contents, and the agent answers using only what's actually in that document. This fits the brief's criteria for genuine agent automation — it's multi-step (read the document, locate the relevant part, produce an answer), it depends on external information supplied at runtime rather than anything baked into the agent beforehand, and it currently requires a human to read the whole document themselves to answer even a single specific question.

It's also a good candidate specifically because the 'external information lookup' step isn't a fixed API or database — it's whatever PDF the user happens to hand the agent, which could be anything from a short story to a technical manual. That variability is exactly what makes naïve keyword-based approaches interesting to test: a system that works cleanly on a synthetic, evenly-formatted test document can still break in genuinely informative ways on a real, professionally-typeset PDF with unusual fonts, abbreviations, and uneven sentence structure — which is exactly what happened during testing here, and is documented in Section 4.

2. Architecture

The agent is a three-node LangGraph state graph: load → retrieve → answer, plus a preloaded variant (retrieve → answer) that skips reloading the PDF on every question, used by the interactive chat interface.

- load — reads the PDF (pypdf), strips control characters left by unusual font encodings, and splits the text into sentence-level chunks.
- retrieve — scores every sentence against the question by keyword overlap (using a 5-character prefix match so minor word-form differences like 'employees' vs 'employs' still count), and keeps the top 3 highest-scoring sentences.
- answer — returns the retrieved sentences directly as the answer.

One limitation disclosed upfront: this environment has no LLM API key, so the 'answer' step is extractive (it returns the actual sentence(s) from the PDF) rather than generative. The exact place a real model call would go is marked ANSWER_WITH_LLM in the code, with the prompt template it would use included as a comment. Everything else — the graph structure, the PDF loading, the retrieval scoring, and the full ReAct trace captured at every step — genuinely executes; nothing here was written after the fact to match a desired result.

3. Evaluation

The agent was tested against a real PDF — Mark Twain's short story A Dog's Tale — rather than a synthetic document, specifically because a real, professionally-typeset PDF exposes formatting issues a clean synthetic test file wouldn't. Seven genuine questions were asked; three are documented in full trace form in the appendix (Section 5).

- Correct and complete: 'How did the dog save the baby?' — retrieved the exact fire-rescue passage from the text.
- Correct: 'What word does the mother dog use as her emergency word?' — retrieved the 'synonymous with supererogation' passage exactly.
- Partially broken: 'What breed was the narrator's father?' — the correct sentence was cut off mid-word ('My father was a St.') due to a sentence-splitting bug described below.
- Missed: 'What was the puppy's name?' — the puppy's name (Robin) is stated only once, in a sentence that doesn't share much vocabulary with the question, so keyword-overlap scoring didn't rank it in the top 3.

Of the 7 questions tested, 4 were answered fully correctly, 2 were partially correct (right general area, but missing or corrupting the specific fact), and 1 missed the correct sentence entirely. This is a realistic, non-tautological result — the test set wasn't curated afterward to make the agent look good.

4. Failure modes found, with hypotheses

Failure 1 — small-caps font encoding produces null bytes (found and fixed)

The book's title page uses a small-caps font that pypdf extracts as literal null-byte characters, which corrupted nearby text. Hypothesis: pypdf's text extraction doesn't always map custom font glyph encodings back to their

intended characters, and small-caps fonts are a common case where this breaks. Fix: replace control/null characters with a space (not delete them outright) during extraction — the first attempt at this fix deleted them entirely, which merged words together across line breaks ('place to' + 'wash' → 'placetowash'), so the fix itself needed a second pass once that regression was caught.

Failure 2 — sentence-splitting breaks on abbreviations (found, not fixed)

The sentence-boundary regex splits on any period followed by whitespace, which incorrectly treats 'St.' (as in 'St. Bernard') as the end of a sentence. This cut the answer to 'What breed was the narrator's father?' off after 'My father was a St.', losing 'Bernard' entirely. Hypothesis: naive regex-based sentence splitting can't distinguish a true sentence-ending period from an abbreviation's period without either a dictionary of known abbreviations or a proper sentence tokenizer (e.g. nltk's punkt model). This wasn't fixed, because a real fix goes beyond the 'very simple and minimal' scope of this assignment — it's documented here instead as a known, explainable limitation rather than hidden.

Failure 3 — keyword-overlap retrieval misses facts stated in unfamiliar wording (found, not fixed)

The puppy's name (Robin) is only revealed once, in the closing burial scene, in a sentence that shares almost no vocabulary with the question 'What was the puppy's name?'. Because retrieval scores purely on keyword overlap, this sentence never rose into the top 3 candidates even though it contains the correct answer. Hypothesis: this is the expected trade-off of using keyword matching instead of real semantic search — a production system would use sentence embeddings and cosine similarity (e.g. via FAISS or Chroma) so that semantically related sentences rank highly even without shared vocabulary.

5. ReAct trace documentation (3 example tasks)

Full Thought/Action/Observation traces for three representative questions, captured directly from real runs — not written after the fact. Two are correct, one shows the abbreviation failure mode described above.

Task 1 (clean success): "How did the dog save the baby?"

Step 1 – Thought: I need to read the PDF and split it into sentences before I can answer anything.

Action: load_and_chunk_pdf | Input: dogs_tale.pdf

Observation: Loaded 119 sentence-level chunks.

Step 2 – Thought: I have the question. I need to find the sentences most relevant to it by keyword overlap.

Action: score_chunk (over all sentences) | Input: 'How did the dog save the baby?'

Observation: Top 3 matching sentence(s) selected out of 119 total.

Step 3 – Thought: I have the retrieved sentences. I'll return them directly as the answer.

Action: return_retrieved_text

Observation: "I reached my head through the flames and dragged the baby out by the waist-band, and tugged it along, and we fell to the floor together in a cloud of smoke..." (correct, complete)

Task 2 (failure — abbreviation bug): "What breed was the narrator's father?"

Step 1 – Thought: I need to read the PDF and split it into sentences before I can answer anything.

Action: load_and_chunk_pdf | Input: dogs_tale.pdf

Observation: Loaded 119 sentence-level chunks.

Step 2 – Thought: I have the question. I need to find the sentences most relevant to it by keyword overlap.

Action: score_chunk (over all sentences) | Input: "What breed was the narrator's father?"

Observation: Top 3 matching sentence(s) selected out of 119 total.

Step 3 – Thought: I have the retrieved sentences. I'll return them directly as the answer.

Action: return_retrieved_text

Observation: "...My father was a St." (cut off – 'Bernard' fell into the next chunk due to the abbreviation-splitting bug)

Task 3 (partial miss — retrieval limitation): "What was the puppy's name?"

Step 1 – Thought: I need to read the PDF and split it into sentences before I can answer anything.

Action: `load_and_chunk_pdf` | Input: `dogs_tale.pdf`

Observation: Loaded 119 sentence-level chunks.

Step 2 – Thought: I have the question. I need to find the sentences most relevant to it by keyword overlap.

Action: `score_chunk` (over all sentences) | Input: "What was the puppy's name?"

Observation: Top 3 matching sentence(s) selected out of 119 total.

Step 3 – Thought: I have the retrieved sentences. I'll return them directly as the answer.

Action: `return_retrieved_text`

Observation: retrieved sentences about the puppy generally, but not the one sentence naming it 'Robin' – that sentence didn't score in the top 3.

6. Production deployment considerations

- Replace keyword-overlap retrieval with sentence embeddings and vector similarity (e.g. FAISS, Chroma), which would likely fix Failure 3 directly, since embeddings capture semantic closeness ('puppy' and 'Robin' appearing in related context) that literal keyword overlap cannot.
- Replace the sentence-splitting regex with a proper sentence tokenizer (e.g. nltk's `punkt`) to fix Failure 2 — `punkt` is trained to recognise common abbreviation patterns and wouldn't split on 'St.' the way a plain regex does.
- Replace the extractive answer step with a real LLM call (the `ANSWER_WITH_LLM` comment in the code marks exactly where), so answers are synthesized in natural language rather than returned as raw sentences — this would also let the agent combine information from multiple retrieved sentences into one coherent answer instead of concatenating them, and would let it say 'I don't know' when the retrieved sentences don't actually answer the question, rather than returning a weak or irrelevant match with unwarranted confidence.
- Add a confidence signal to each answer (e.g. the top retrieval score), so the agent — or a downstream UI — can flag low-confidence answers rather than presenting a weak match with the same certainty as a strong one. Failure 3 in particular would benefit from this: a low top-score should visibly warn the user that the answer might be incomplete, rather than silently returning the best available (but wrong) match.
- Handle multi-page and multi-document PDFs more deliberately — the current chunking treats the whole PDF as one flat list of sentences with no page or section metadata, so a production version answering questions over long or structured documents (e.g. a 200-page manual) would need to track and surface page numbers alongside each answer for traceability.